11/03/2025

Goals:

1. ~~Add camera to robots in Mujoco ur5gripper sim~~
2. Make RGB information accurate to intel realsense d435 camera sensor
3. Add Depth sensor for realsense d435
4. Add Lidar point cloud sensor

Progress:

Added a camera to the robot model where the realsense is located, allowing the point of view of the robot to be rendered.

```
<camera name="Robot_Front" pos="0.4103 -0.94641e-05 0.174" quat="0.5 0.5 -0.5 -0.5" mode="fixed" />
```

https://www.youtube.com/watch?v=wft9hBcDyk4

Problems:

Front Camera clips with what is being rendered causing parts of the simulation to disappear in the camera's view.

~~Point cloud~~ Depth image functionality is not a feature mujoco has normally, but can be done using openGL

Next Week:

1. Learn how mujoco uses openGL
2. Get data from mujoco using openGL
3. Add global view camera
    a. For extracting information
4. Document process for adding depth to Mujoco

18/03/2025

Goals:

5. ~~Add camera to robots in Mujoco ur5gripper sim~~
6. Make RGB information accurate to intel realsense d435 camera sensor
7. Add point cloud sensor for realsense d435
8. Add Lidar point cloud sensor

Progress:

none

Problems:

N/A

Next Week:

25/03/2025

Goals:

1. Learn how mujoco uses openGL
2. Get data from mujoco using openGL
3. Add global view camera
    a. For extracting information
4. Document process for adding depth to Mujoco

Progress:

Found a tutorial by google deep mind for a library they use for exporting depth data via openGL:

https://colab.research.google.com/github/google-deepmind/mujoco/blob/main/python/tutorial.ipynb

Got nvidia drivers working because library uses nvidia-smi (system management interface)

Problems:

Could not get the depth map program to run. The depth map python programs failed to run due to a library called mediapy. This library lets the program run in a jupyter notebook but seems to be causing problems when running on my laptop.

I also found a python based renderer for mujoco using OpenGL https://github.com/rohanpsingh/mujoco-python-viewer that I can try yo get the depth map from following this guide https://gist.github.com/lucasplus/06f0cc6957851fc28c6c201bf287d89f#file-ksim-cpp-L469

Next Week:

1. Get data from mujoco using openGL
2. Continue work on depth map
3. Add global view camera
    a. For extracting information
4. Document process for adding depth to Mujoco

01/04/2025 April Fools!

Goals:

1. Learn how mujoco uses openGL
2. ~~Get data from mujoco using openGL~~
3. Add global view camera
    a. For extracting information
5. ~~Document process for adding depth to Mujoco~~

Progress:

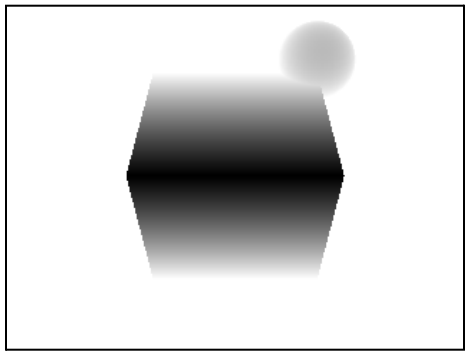
Used Google Deepminds “dm_control” library to access render data from Mujoco’s simulation without directly using openGL.

```
def render_depth(model):
      physics = mujoco.Physics.from_xml_string(model)
      pixels = physics.render()
```

Using the mujoco.Physics method I was able to get per-pixel depth data:

```
# depth is a float array, in meters.
depth = physics.render(depth=True)
```

This method was able to provide this image of a simple scene and should be able to create video streams as well for sensor data.



Problems:

This method relies on dm_control and I am not sure if it can be used as a drop in replacement for mujoco despite it containing mujoco.

Next Week:

5. Continue work on depth map
6. Add global view camera RGB-D
    a. For extracting information
7. Document process for adding depth to Mujoco
8. Get Depth video
9. dm_control tutorial/demo

08/04/2025

Goals:

1. Continue work on depth map
2. Get depth video
3. dm_control tutorial/demo

Progress:

Read google deepmind's tech report on the dm_control library

Wrote " Google Deepmind dm_control basics " report

Problems:

None last week

Next Week:

10. Continue work on depth map
~~11. Add global view camera RGB-D~~
    ~~a. For extracting information~~
~~12. Document process for adding depth to Mujoco~~
13. Get Depth video
14. Generate Point Cloud

22/04/2025

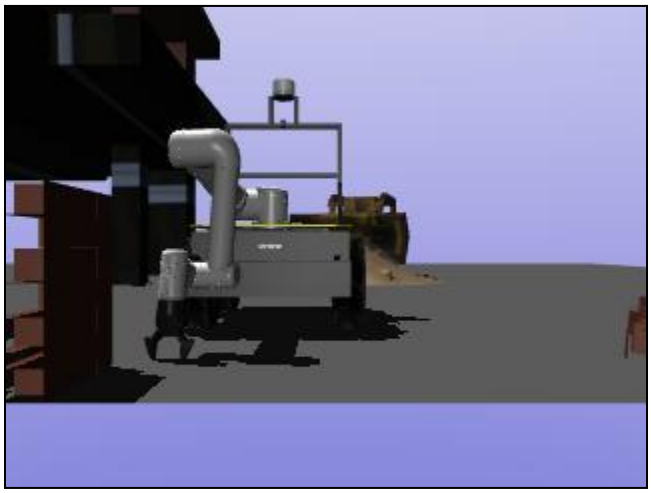
Goals:

1. ~~Create depth video~~
2. ~~Make it pull from any MJCF file~~
3. Generate point cloud

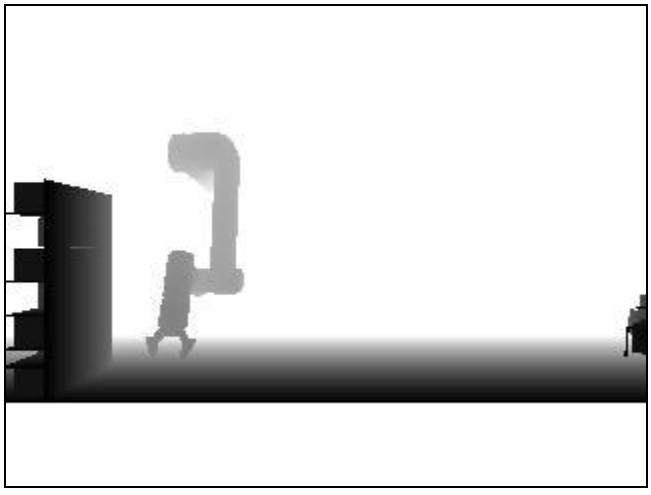
Progress:

Created multiple render methods using dm_control's mujoco tools to get visual data from Mujoco's OpenGL renderer.

def render_rgb(model):



def render_depth(model):



def render_rgb_video(model): (https://youtu.be/5rU6mnNu85Q)
def render_depth_video(model): (https://youtu.be/ens96xldosk)

Demo of switching methods: (https://youtu.be/jc8513I5F7c)

Problems:

Performance (frame buffer might help)
Clipping with nearby objects

Next Week:

1. Continue work on depth video
2. Generate Point Cloud
3. Absolute depth

29/04/2025

Goals:

1. ~~Continue work on depth video~~
2. ~~Generate Point Cloud~~
3. ~~Absolute depth~~

Progress:

Added ffmpeg frame exporting and true depth point cloud generation. The frames are saved to a folder which then can be read by a ffmpeg to mp4 script I also created. The point cloud is formatted and is saved as a dataframe with x,y,z and rgb information but the rgb is unused.

The frames being saved to a folder rather than being displayed as they are rendered has made render times very fast but U have not measured the impact.

Problems:

Camera clipping is still not addressed

Pointcloud is not saved as time stamped frames but only printed to terminal output

Output resolution takes some time to change as the frame buffer needs to be changed too

Next week:

Meet with Mohsen this week to discuss next steps

Visual Depth map
Multi camera (overhead)
Research updating mujoco with ROS1 (ROS1 run on husky)
Measure point cloud generation time if it's easy

06/05/2025

Goals:

1. Research updating mujoco with ROS1 (ROS1 run on husky)
2. ~~Measure point cloud accuracy~~
3. ~~Add overhead point cloud camera~~
4. Visualize point cloud

Progress:

Looked into using .ply format for saving point cloud data, and found that there are simple to use libraries that can handle the conversion. The .ply format is common and can be read by most 3d programs to allow for debugging and validation of point cloud data.

Tested accuracy of depth map by placing a thin box 0.5m away from the overhead camera that was added to the scene. The box has a thickness of 0.002m meaning the planar distance should be (0.5 - 0.001) = 0.499m. The depth map render was run collecting data from the overhead camera as shown in the picture. The point cloud data that was returned is below:

```
——————————————————————————————————————————————
                x           y      z
0      -122.921776 -69.143499  0.499
1      -122.537645 -69.143499  0.499
2      -122.153515 -69.143499  0.499
3      -121.769384 -69.143499  0.499
4      -121.385254 -69.143499  0.499
...            ...         ...    ...
76795    -1.920653  -0.288098  0.499
76796    -1.536522  -0.288098  0.499
76797    -1.152392  -0.288098  0.499
76798    -0.768261  -0.288098  0.499
76799    -0.384131  -0.288098  0.499
——————————————————————————————————————————————
```



For all frames the z values of all pixels was the expected 0.499m showing that the “Truedepth” render does not distort depth measurements in the point cloud data.

Problems:

My first approach for rendering

Next Week:

1. Research Mujoco exporting data to ROS
2. Measure point cloud generation speed (in frames per second)
3. Visualize point cloud

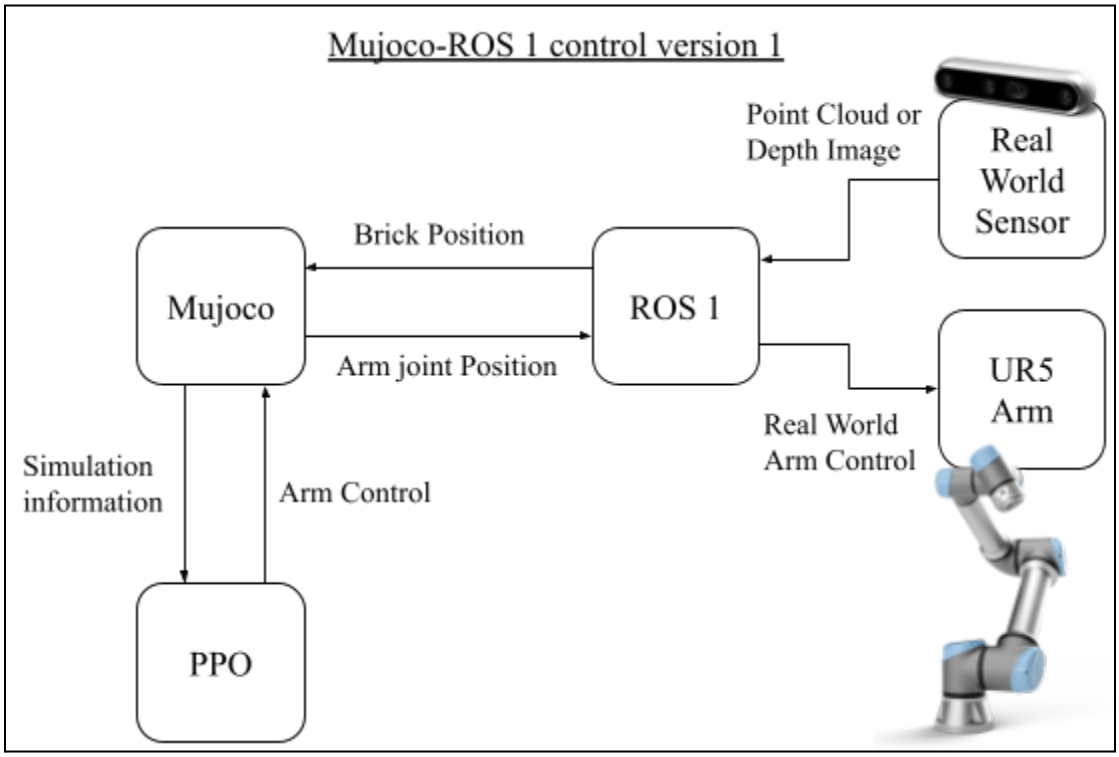
06/20/2025

Goals:

1. Research Mujoco exporting data to ROS
2. Measure point cloud generation speed (in frames per second)
3. Visualize point cloud

Progress:

Planned to take real world point-cloud data and process it to get brick positions that would be loaded into mujoco each timestep where the PPO model will inference based on the simulation state. The generated positions will be published by the Mujoco node and then using the MovieIT ROS package the arm will go to the position.



Also gained access to Lambda PC to begin working on ROS packages to test models in the real world.

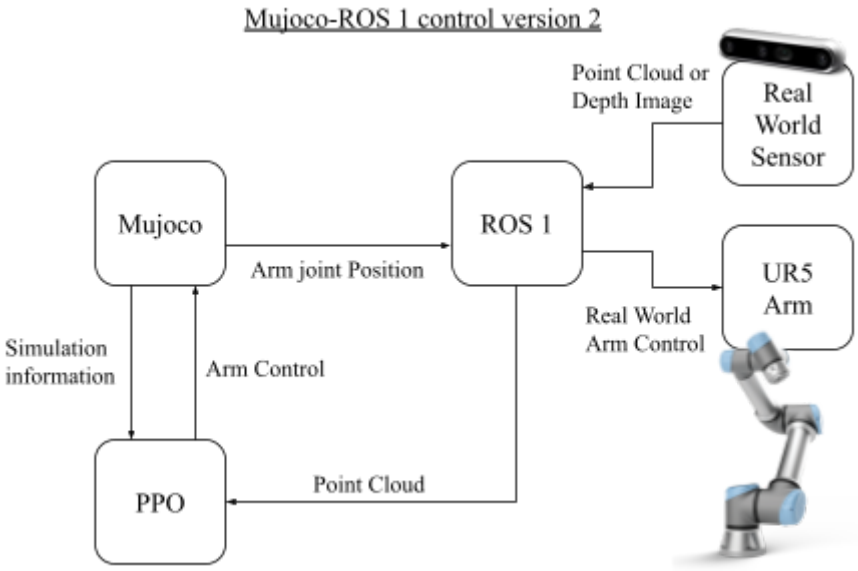
Problems:

Next Week:

1. Create Mujoco ROS package
2. Control UR5 with mujoco control data
3. Measure point cloud generation speed (in frames per second)
4. Visualize point cloud

~~5. Image brick with realsense~~

6. Get brick position from pointcloud



06/27/2025

Goals:

1. Develop ROS1 package for mujoco integration
2. Control UR5 with mujoco control data
3. ~~Measure point cloud generation speed (in frames per second)~~
4. ~~Visualize point cloud~~
5. Get brick position from point cloud

Progress:

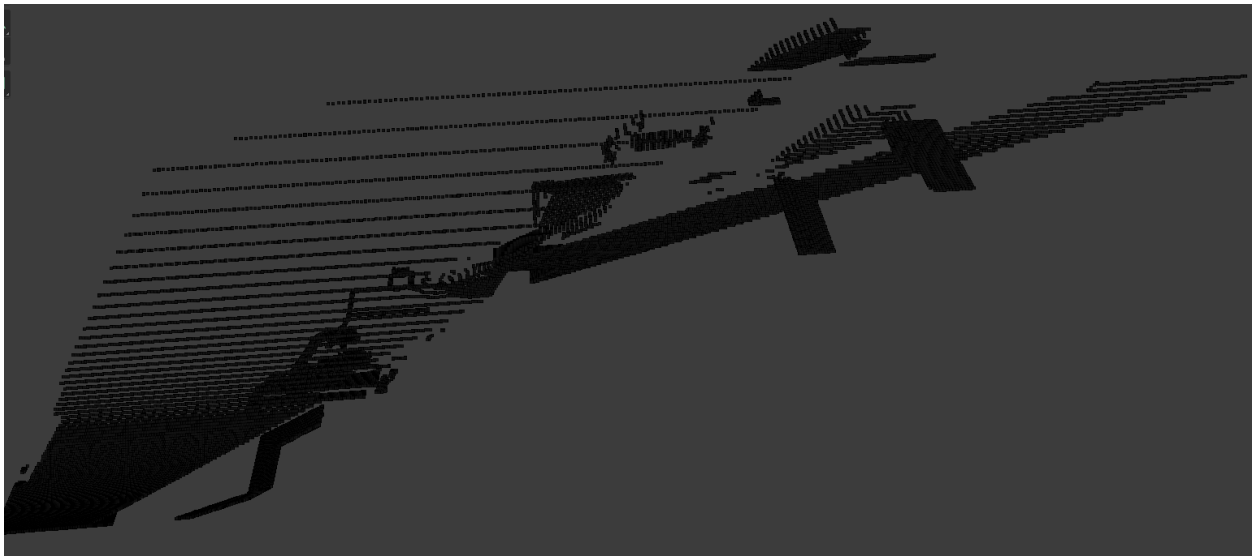
This week I looked at MJCF data

I also took time to measure the performance of the point cloud renderer and found that the frame generation time was about 27 frames per second or 0.0355 seconds per frame. The hardware the code is run on had an effect on the frame generation speed, where the more powerful the hardware the faster point clouds were generated.

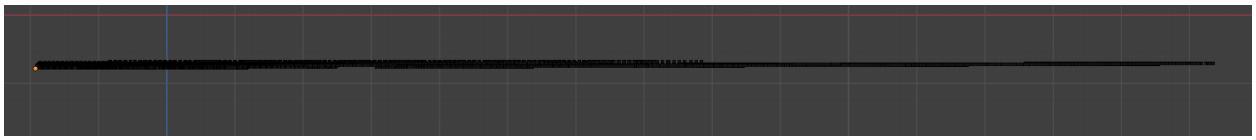
Trial	Seconds Per Frame	Frames Per Second
1	0.03639456918162684	27.4766269
2	0.03585639692121936	27.8890264
3	0.03464024297652706	28.8681578
4	0.037195336434148976	26.8850909
5	0.03693565245597593	27.0741122
Average	0.03620443959	27.6209219441

Problems:

Pointcloud is not saved in PLY format correctly



Here we can see that the x y and z data is getting mixed in the PLY file and the z data is scaled wrong and below we are looking at the points from the side and can see the depth data is captured but at the wrong scale.



Next Week:

1. Develop ROS1 package for mujoco integration
2. Control UR5 with mujoco control data
3. Get brick position from point cloud
4. Fix point cloud saving in PLY format

On lambda, make folder under my name for project

AIPM
Minecraft tutorial copying build on their own
Same human project manager for each team
3 rounds of human studies